



الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

Service Provider Update Instructions

Version 1.0 -- ID Card Toolkit SDK v3.1.x

May 2026

CONFIDENTIAL

Table of Contents

Revision History	3
1 Introduction	4
2 Windows SDK (C / C++)	4
2.1 Update steps	4
2.2 Additional configuration	4
3 Windows SDK (.NET)	4
3.1 Update steps	4
3.2 Additional configuration	4
4 Java SDK (Windows desktop)	5
4.1 Update steps	5
4.2 Additional configuration	5
5 Web / JWS SDK	5
5.1 Update steps	5
5.2 Additional configuration	5
6 Emirates ID Card Toolkit Service (Windows MSI)	5
6.1 Update steps	5
6.2 Additional configuration	6
7 Android SDK	6
7.1 Update steps	6
7.2 Additional configuration	6
7.3 Migrating from an earlier Toolkit version	7
7.3.1 Toolchain prerequisites	7
7.3.2 Plugin AARs from older releases will not load	8
7.3.3 AGP 8.x rejects hyphenated package= attributes	8
7.3.4 Gson dependency relocation	8
7.3.5 NFC reader changes (Android 12+ / targetSdk 31 and newer)	8
8 iOS SDK	9
8.1 Update steps	9
8.2 Additional configuration	9
9 Linux SDK – C / C++	9
9.1 Update steps	9
9.2 Additional configuration	10
10 Linux SDK – Java	10
10.1 Update steps	10
10.2 Additional configuration	10

Revision History

Version	Date	Description of Changes
1.0	May 2026	Initial single-point-of-reference upgrade document for the v3.1.x series. Covers v3.1.0 through v3.1.6.

1. Introduction

This document lists the per-SDK steps to upgrade to ID Card Toolkit v3.1.x (v3.1.0 through v3.1.6) and the additional configuration each SDK requires on the consumer side.

There are no breaking source-level API changes in v3.1.x; existing application code recompiles against the new SDK without modification. Replace the SDK binaries, follow any additional-configuration step for your SDK, then rebuild and redeploy.

2. Windows SDK (C / C++)

2.1. Update steps

1. Replace `EIDAToolkit.dll` and link against the updated `EIDAToolkit.lib`.
2. **Plugin placement (required).** At runtime the toolkit loads each plugin DLL named in `config_pg` from the same directory as `EIDAToolkit.dll`. It resolves the bare `plugin_library_path` file name against `EIDAToolkit.dll`'s own directory; on Windows there is no working-directory or `PATH` search and no configuration option for this. For every reader you support, copy the entire contents of the matching `plugins/<Vendor>/<arch>/` folder (the plugin DLL plus its accompanying dependency DLLs) into the directory that holds `EIDAToolkit.dll`. This includes `PCSCLib.dll` from `plugins/PCSC/<arch>/` for smartcard access. The `quickstart/<arch>/` folder is a working example of this flat layout. Do not keep the `plugins/<Vendor>/<arch>/` nesting at the deployment location.
3. Rebuild.

2.2. Additional configuration

None.

3. Windows SDK (.NET)

3.1. Update steps

1. Replace `IDCardToolkit.dll` and the native `EIDAToolkit.dll`.
2. **Plugin placement (required).** `IDCardToolkit.dll` calls into the native `EIDAToolkit.dll`, so the same rule as the Windows SDK (C / C++) above applies: the plugin DLLs and their dependency DLLs must sit in the same directory as `EIDAToolkit.dll`. The `plugins/<Vendor>/<arch>/` nesting is the SDK packaging layout only, not the runtime layout.
3. Rebuild.

3.2. Additional configuration

None.

4. Java SDK (Windows desktop)

4.1. Update steps

1. Replace `EIDAToolkit.jar` with the Windows build (it embeds the native libraries for both 32-bit and 64-bit Windows JVMs).
2. Rebuild.

4.2. Additional configuration

None on Windows. The JAR is self-contained for both 32-bit and 64-bit Windows JVMs.

The `EIDAToolkit.jar` is platform-specific: its embedded native libraries are per operating system. For a Linux desktop deployment use the Linux Java SDK build of the JAR and follow its requirements – see *Linux SDK – Java* below.

5. Web / JWS SDK

5.1. Update steps

1. Replace the JWS jar on your web server:
 - `IDCardToolkitService.jar` for Sagem MSO 1350 reader deployments.
 - `IDCardToolkitService-Nexus.jar` for Richmond Nexus reader deployments (new in v3.1.4).
2. Replace `eidatoolkit.js`.
3. Rebuild and redeploy the web application.
4. Restart the JWS launcher on each end-user endpoint.

Applet SDK (Nexus). Deployments that use the browser applet SDK rather than the JWS launcher now have a Nexus variant, `PublicDataApplet-Nexus.jar`, for Richmond Nexus reader deployments (new in v3.1.6). Replace the applet jar in your web application the same way as the standard `PublicDataApplet.jar`.

5.2. Additional configuration

None.

6. Emirates ID Card Toolkit Service (Windows MSI)

6.1. Update steps

1. Run the new MSI installer. It performs an in-place upgrade and preserves the existing service configuration.
2. If in-place upgrade is not possible, uninstall "Emirates ID Card Toolkit Service" via **Windows Settings > Apps** first, then run the new MSI.

New in v3.1.6. The installer now bundles the Richmond Nexus device plugin (for the ICP National Card Reader / Dalil AEGIS PRO 4) on both 32-bit and 64-bit installs. No additional steps are required to enable Nexus reader support after upgrading.

6.2. Additional configuration

None.

7. Android SDK

7.1. Update steps

1. Replace EIDAToolkit.aar and the relevant plugin AAR files in app/libs/ (or your dependency directory).
2. Rebuild via Gradle.

7.2. Additional configuration

Gson dependency. EIDAToolkit.aar references com.google.gson.* classes in its digital-signature data models but does not bundle Gson itself. The path you use to consume the AAR determines whether you need to declare Gson explicitly:

Consumption path	What to do
implementation project(':EIDAToolkit') (sub-project shim)	Add 'com.google.code.gson:gson:2.10.1' to the shim project's default configuration so the dependency is transitively propagated. The sample app's samples/android/EIDAToolkit/build.gradle shows this pattern.
implementation files('libs/ EIDAToolkit.aar') (raw file dependency)	File-based dependencies have no metadata to propagate. Add implementation 'com.google.code.gson:gson:2.10.1' directly to your app's build.gradle.

Without this, Android Gradle Plugin fails the first build with a DexingNoClasspathTransform error referencing com.google.gson.*.

Plugin packaging options. The following settings are required in your app module's android { } block for plugins to load correctly at runtime:

- packagingOptions { doNotStrip "**/*.so" } – required for plugins to be accepted by the SDK loader.
- packagingOptions { jniLibs { useLegacyPackaging = true } } – required for the SDK to locate plugin native libraries at runtime.
- packagingOptions { pickFirst 'lib/arm64-v8a/libc++_shared.so' } and pickFirst 'lib/armeabi-v7a/libc++_shared.so' – resolves libc++ runtime conflicts when two or more plugin AARs ship libc++_shared.so built against different NDK versions.
- defaultConfig { multiDexEnabled true } – the toolkit + plugin set exceeds the 64K method limit.

xmlsec2-release module. Required whenever your app uses any digital-signature or signature-verification API (PadesSigningContext, VerificationContext, etc.). Include ':xmlsec2-release' in settings.gradle and add implementation project(':xmlsec2-release') to your app's build.gradle. Omitting it compiles cleanly but throws ClassNotFoundException for org.apache.xml.security.* at signing time.

AndroidManifest.xml application attributes. Two attributes must be present on the <application> tag for the Toolkit to find its plugin libraries and configuration files at runtime:

```
<application
...
    android:extractNativeLibs="true"
    android:requestLegacyExternalStorage="true">
```

- **extractNativeLibs="true"** – Android Gradle Plugin 4.1 and newer set this attribute to `false` by default and keep native libraries compressed inside the APK. The Toolkit loads its smart-card and biometric plugins through `dlopen()` against a filesystem path; compressed-in-APK libraries return "library not found" at load. The sibling `packagingOptions { jniLibs { useLegacyPackaging = true } }` Gradle setting is required for the same reason – both must agree.
- **requestLegacyExternalStorage="true"** – Android 10 (API 29) introduced scoped storage. If the Toolkit or your app reads `config_*` files from external-storage paths (e.g. `/sdcard/...`), the flag is required for direct path access to keep working at `targetSdkVersion = 29`. From Android 11 (API 30) onward the flag is honoured only by apps with a compatible `targetSdkVersion`; the long-term recommendation is to deploy configuration files into the app's private storage (`getFilesDir()`) instead.

Other manifest items worth reviewing during an upgrade. Depending on the feature set your app exercises, you may also need:

- Storage permissions (`READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`, and on some deployments `MANAGE_EXTERNAL_STORAGE`) if your app reads or writes configuration / signed-output files from shared storage. From Android 13 (API 33) the `READ_EXTERNAL_STORAGE` permission is replaced by per-type `READ_MEDIA_IMAGES` / `READ_MEDIA_VIDEO` / `READ_MEDIA_AUDIO` for media files.
- `android:usesCleartextTraffic="true"` (or an equivalent `network_security_config.xml`) if your Validation-Gateway or test endpoint is reached over plain HTTP rather than HTTPS.
- `android:largeHeap="true"` if your app embeds biometric ML models alongside the Toolkit's own and routinely hits the default per-app heap limit.

7.3. Migrating from an earlier Toolkit version

Service providers upgrading an existing Android integration from an earlier Toolkit release (v3.0.x or older) should also account for the following.

7.3.1. Toolchain prerequisites

The v3.1.x Android sample is built and verified with:

Component	Minimum / verified version
Java / JDK	Java 11 (<code>sourceCompatibility</code> / <code>targetCompatibility</code> <code>VERSION_11</code>)
Android Gradle Plugin	8.2.1
Gradle wrapper	8.14.2 (the bundled <code>gradlew</code> downloads this on first run)
<code>compileSdkVersion</code>	35
<code>targetSdkVersion</code>	34 (raise as Google Play raises the floor)
<code>minSdkVersion</code>	28 (Android 9.0)

If your existing project still uses AGP 7.x or earlier, upgrade Gradle and AGP first, then swap in the new AARs. Mixing v3.1.x AARs with an AGP 7.x build occasionally surfaces obscure DEX-merge failures that disappear once AGP is current.

7.3.2. Plugin AARs from older releases will not load

Every smart-card / biometric plugin shipped with the Toolkit is verified at load time against a SHA256 hash baked into the toolkit library. Plugin AARs taken from a prior Toolkit release will not load against v3.1.x and will be reported as tampered. Always use the plugin AAR set that matches the Toolkit version you are upgrading to. If you maintain a private vendored copy of a plugin AAR, rebuild it against the v3.1.x source before shipping.

7.3.3. AGP 8.x rejects hyphenated package= attributes

Plugin AAR manifests produced by older release scripts contained hyphenated package= attributes (for example `ae.emiratesid.idcard.toolkit.plugin.idscreen-plugin`). AGP 8.x rejects these as invalid Java identifiers. The plugin AARs that ship in v3.1.x already use hyphen-free package names. If your project includes a locally-vendored older AAR, either replace it with the v3.1.x version or patch the manifest in place to remove the hyphen before depending on it.

7.3.4. Gson dependency relocation

Older releases required service providers to add `com.google.code.gson:gson` to their own `app/build.gradle`. In v3.1.x the AAR-shim form (`implementation project(':EIDAToolkit')`) carries Gson as a transitive dependency, so the explicit line is no longer required for shim consumers. Raw-file consumers (`implementation files('libs/EIDAToolkit.aar')`) still need the explicit declaration – see the table above.

7.3.5. NFC reader changes (Android 12+ / targetSdk 31 and newer)

If your application enables NFC reading and is moving its `targetSdkVersion` to 31 (Android 12) or higher, two changes to the host activity are required.

Mandatory `PendingIntent` mutability flag. Android 12 made it illegal to construct a `PendingIntent` without explicitly declaring its mutability. The toolkit's NFC `ReadNFCActivity` and `OperationActivity` patterns build a foreground-dispatch `PendingIntent`; the fourth (flags) argument must now carry either `FLAG_MUTABLE` or `FLAG_IMMUTABLE`.

Old form (works on Android up to 11, crashes on Android 12+ when your `targetSdkVersion` is 31 or newer):

```
mPendingIntent = PendingIntent.getActivity(  
    this, 0,  
    new Intent(this, getClass()).addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP),  
    0);  
                                // ← no mutability flag
```

New form (works on every Android version the toolkit supports):

```
mPendingIntent = PendingIntent.getActivity(  
    this, 0,  
    new Intent(this, getClass()).addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP),
```



```
PendingIntent.FLAG_MUTABLE);    // FLAG_IMMUTABLE if your intent never needs to  
be modified
```

Use `FLAG_MUTABLE` if Android's NFC subsystem needs to add the discovered tag to the intent at delivery time (the foreground-dispatch case in the sample). Use `FLAG_IMMUTABLE` if your activity already carries everything it needs. Passing 0 on Android 12+ with `targetSdkVersion >= 31` crashes immediately with `IllegalArgumentException: PendingIntent must be created with FLAG_IMMUTABLE or FLAG_MUTABLE`.

The `FLAG_MUTABLE` constant is defined in `android.jar` from API 31 onward; referencing it requires `compileSdkVersion >= 31`. The bit it represents (`0x02000000`) is unrecognised by older Android runtimes, so the same source code runs safely on every device down to `minSdkVersion 28` – no `Build.VERSION.SDK_INT` guard is required.

Reader-mode vs foreground-dispatch. The sample uses `enableForegroundDispatch` for backward compatibility. For new self-service / kiosk deployments, prefer `enableReaderMode` – it gives the toolkit explicit control over which tag technologies are discovered and avoids the activity-restart cycle that foreground dispatch triggers on every tag detection. This is a code-side choice; no manifest change is required.

8. iOS SDK

8.1. Update steps

1. Replace `IDCardToolkit.framework` and the relevant plugin frameworks in your Xcode project.
2. Rebuild via Xcode.

8.2. Additional configuration

Embed & Sign. For each replaced framework, set **Embed & Sign** in **Target > General > Frameworks, Libraries, and Embedded Content**. Frameworks set to **Do Not Embed** will link successfully but the app will crash at startup with "Library not loaded" against the framework path.

ext/ directory. The SDK bundle includes an `ext/` directory next to `lib/` containing static OpenSSL and xmlsec1 dependencies (`ext/openssl/` and `ext/xmlsec/lib/iOS/libxmlsec.a`). `IDCardToolkit.framework` links against these at build time. Preserve the `ext/` tree at the same relative path as the framework, or update your Xcode project's `FRAMEWORK_SEARCH_PATHS` and the `libxmlsec.a` file reference to point at your own copy. Omitting `ext/` produces undefined-symbol errors at link time (`_xmlSecInit`, `_SSL_CTX_new`, etc.).

For a complete integration walkthrough (Xcode project setup, framework embedding, and NFC reading), see the *ID Card Toolkit iOS Integration Guide* included in the documentation set.

9. Linux SDK – C / C++

9.1. Update steps

1. Replace `libEIDToolkit.so` and link against the updated library.

2. **Plugin placement (required).** At runtime the toolkit loads each plugin named in `config_pg` from the same directory as `libEIDAToolkit.so`. It resolves the bare `plugin_library_path` file name against `libEIDAToolkit.so`'s own directory (located via `dladdr`), not via `LD_LIBRARY_PATH` or `rpath`. Place `libPCSCLib.so` and the relevant plugin `.so` files (plus any vendor dependency libraries they require) in the same directory as `libEIDAToolkit.so`.
3. Rebuild.

9.2. Additional configuration

PC/SC system package. The toolkit links against pcsc-lite. Install it via the deployment endpoint's package manager:

- Debian / Ubuntu: `apt install libpcsclite1`
- RHEL / CentOS / Fedora: `yum install pcsc-lite-libs`

10. Linux SDK – Java

10.1. Update steps

1. Replace `EIDAToolkit.jar`.
2. Rebuild.

10.2. Additional configuration

Same PC/SC system package requirement as the Linux C / C++ SDK above.